



SolarSandbox

Members:

Clément Desjonqueres

Pierre Quintart

Arthur De Pastors

Arthur De Carvalho

Adrien Bornet

Bruno Factor

Gitlab:

<https://gitlab-cw3.centralesupelec.fr/objective-26.5/solarsandbox>

What's SolarSandbox?

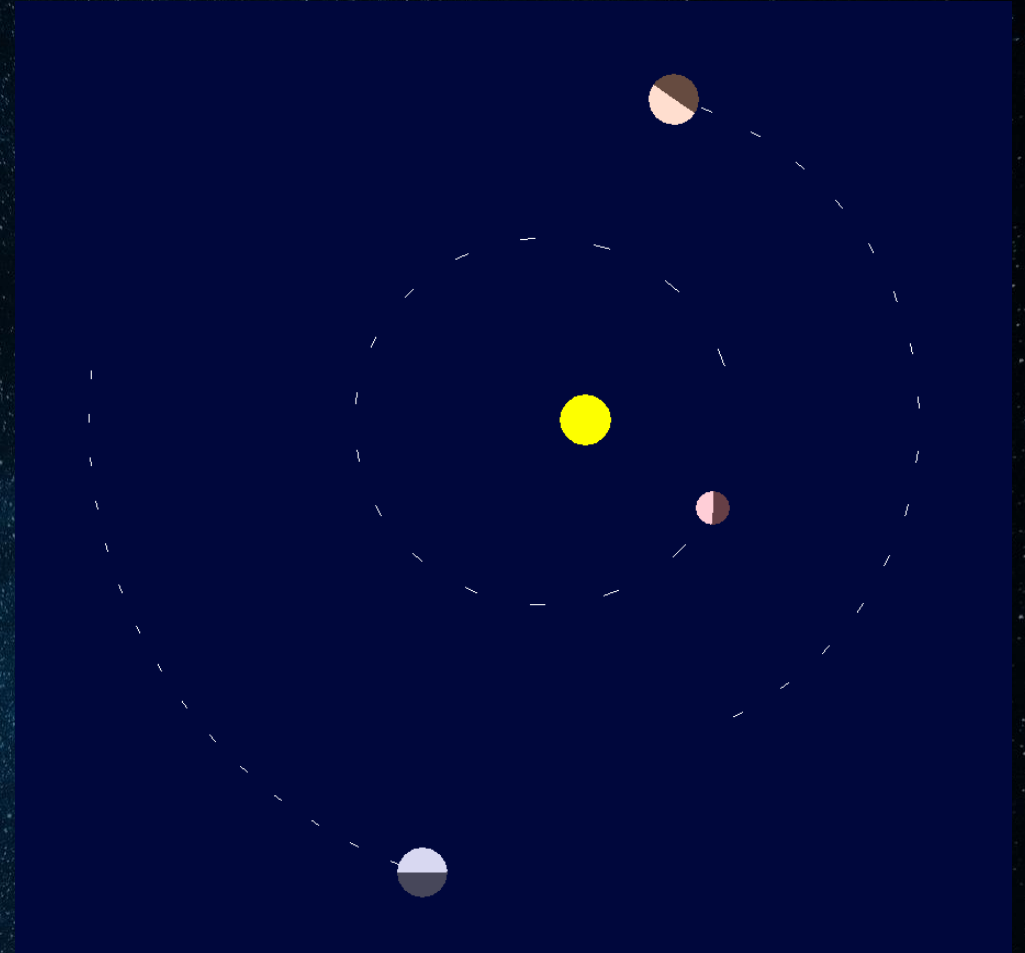
To simulate gravitational interactions!

It allows you to :

- See the solar system in motion
- Modify the characteristics of celestial bodies
- Add or remove celestial bodies

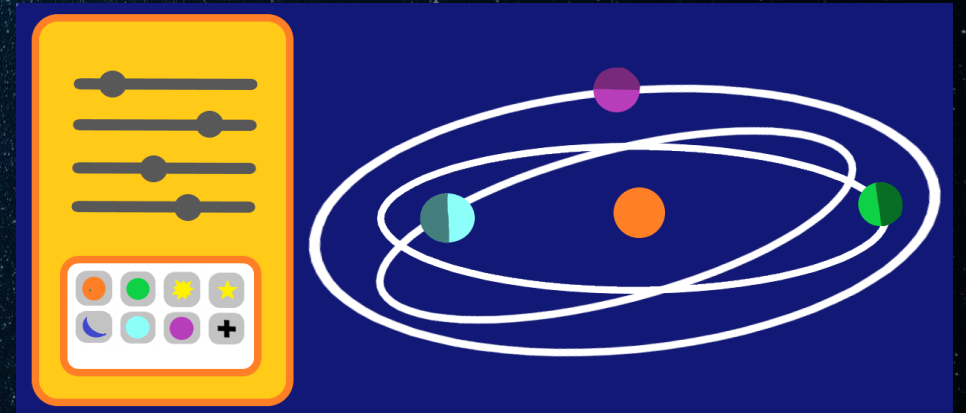
For who ?

The application is primarily for **educational use**, from primary school to university, but thanks to its ease of use, anyone can have fun creating their own 3-sun solar system!



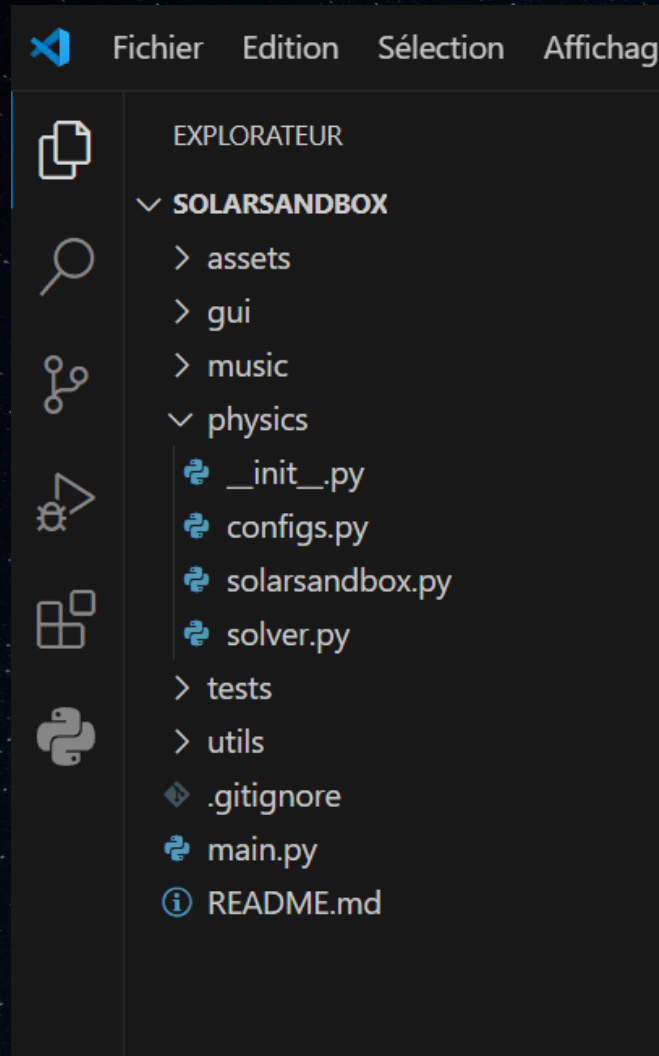
Which features ?

- **Simulate** the gravitational interactions between bodies
- Enable users to **observe** the solar system:
 - **Display information about** the selected celestial body
 - Enable the user to **navigate** (zoom in and out) in the solar system)
- Enable the user to **modify** the properties of a celestial body
- Allow the user to **add** or **delete** a celestial body
- Allow users to **choose their initial configuration**
- **Be aesthetically pleasing!**
 - Take care with the user interface
 - Display the **trajectories** and shadows of the planets



initial project sketch

Code's structure



Assets : images and music

configs: preset configurations

solver : using RK45 to solve for trajectories

(Runge-Kutta method of order 4-5)

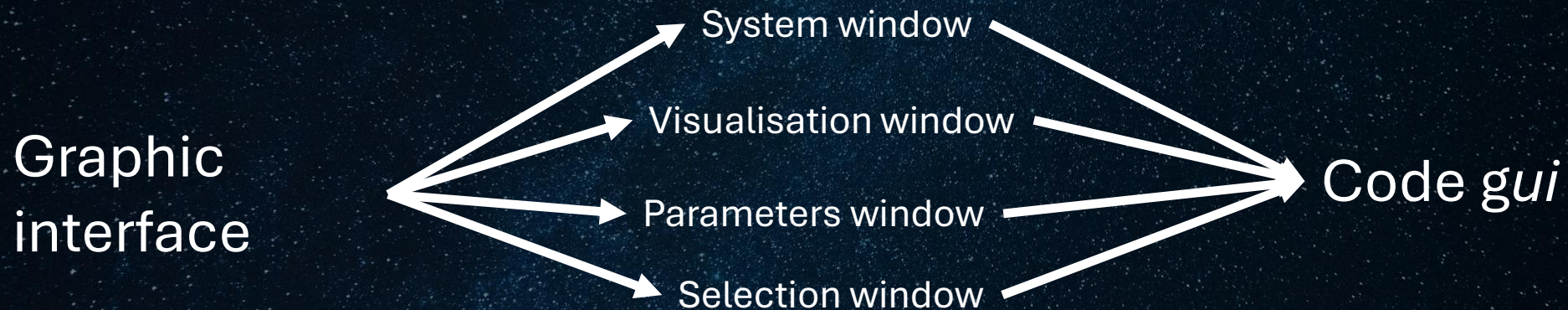
Solarsandbox : main code defining Stars, Planets, calculating temperatures, colors,...

Gui : interface code displaying those objects

Work's division

Physics → Code solver

Definition → Code solarsandbox



Physics

Integration Method:

- We use the **Runge-Kutta 45** method (RK45) to solve the equations of motion for each celestial body.

SimulationSolver Class:

- The SimulationSolver class manages the time evolution of the solar system, computing the movement of celestial bodies.

Time Bound: Defines the maximum simulation time (e.g., 700 days).

Time Step: Specifies the interval at which the system updates (e.g., 1 hour per step).

Key Features:

Body Reset: Each celestial body can be reset to its initial state and reattached to the solver.

Collision Detection: The code checks for collisions between bodies and handles them accordingly.

Solver Step: Each body's position and velocity are updated at each time step, and collisions are managed in real-time.

Collision Management:

- When a collision is detected between bodies, the simulation responds by managing the resulting interactions.

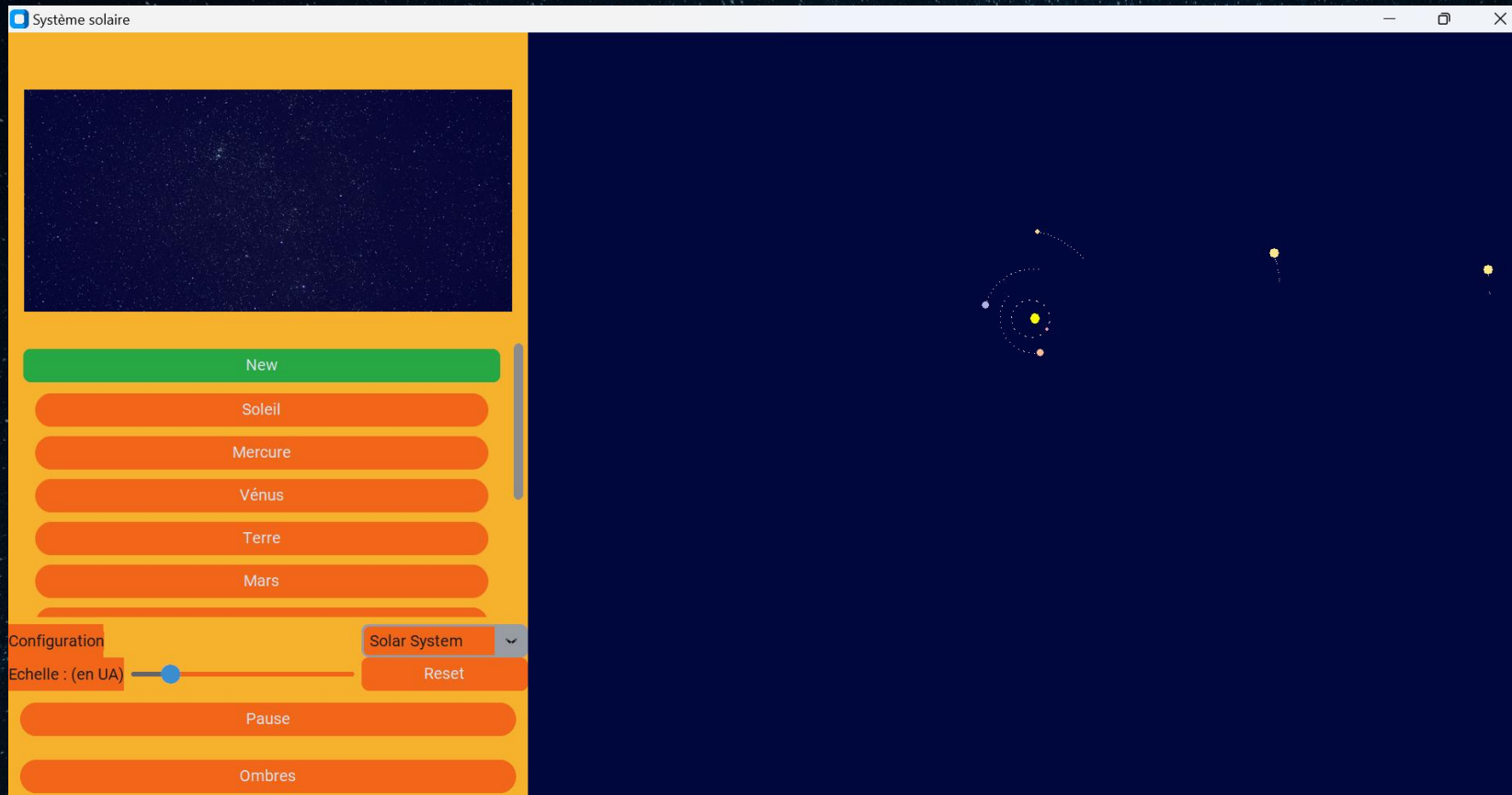
Definition

- **Celestial Bodies:** Planets and Stars with properties like mass, radius, and position.
- **Gravity Interactions:** Gravitational forces calculated between bodies.
- **Graphical Interface:** Visualization using Tkinter and customTkinter.
- **Collisions Management:** Merging smaller bodies with larger ones.
- **Planetary Properties:** Albedo, atmosphere type, and equilibrium temperature calculations.
- **Simulation Features:** Trajectory tracking and dynamic updates.

Graphic interface

Various windows: general window, selection window, view/edit window and creation window

OBJECTIVE: a simple, interactive interface!



Windows

Name: Earth
Mass: $5.97\text{e}+24$ kg
Radius: 6371 km
Equilibrium temperature: 213.95 K



New

Sun

Mercury

Venus

Earth

Mars

Jupiter

Saturn

Uranus

Configuration

Scale : (UA)

Solar System

Reset

Pause

Shadows

Name: Earth
Mass: $5.97\text{e}+24$ kg
Radius: 6371 km
Equilibrium temperature: 213.06 K



Name Earth

Mass $5.97\text{e}+24$

Radius 6371

Albedo 0.3

Atmosphere H₂

New

Sun

Mercury

Venus

Earth

Configuration

Scale : (UA)

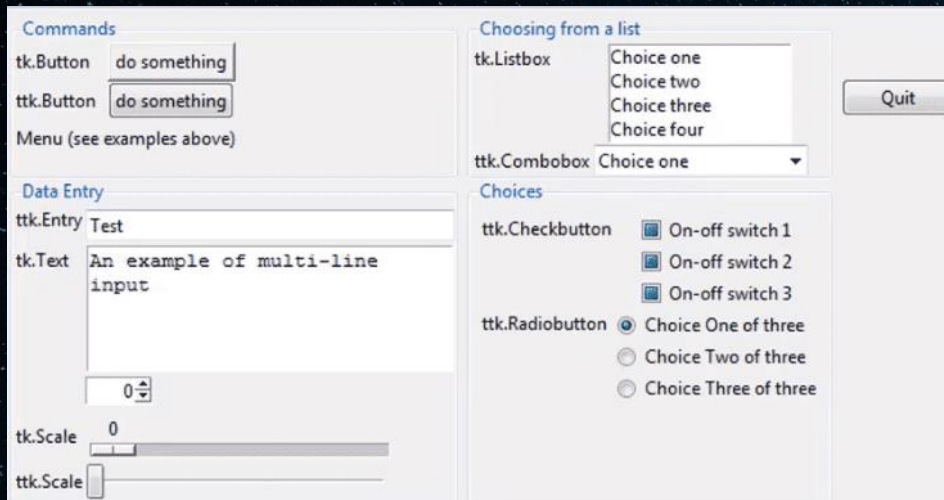
Solar System

Reset

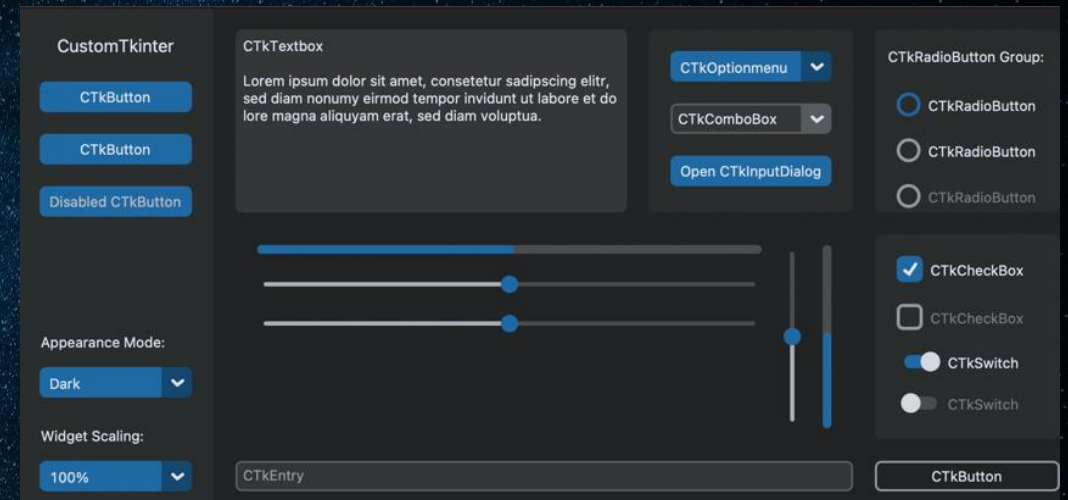
Pause

Shadows

Enhanced visual rendering with the CustomTkinter module



widgets tkinter



widgets customtkinter

Windows

Name: Sun
Mass: 1.989×10^{30} kg
Radius: 696340 km
Lumino

Ajouter un nouvel astre

Name:

Weight (kg):

Radius (m):

Semi-major axis:

Trajectory radius:

Default

Add

Cancel

Ajouter un nouvel astre

Name:

Weight (kg):

Radius (m):

Position (x, y) (m):

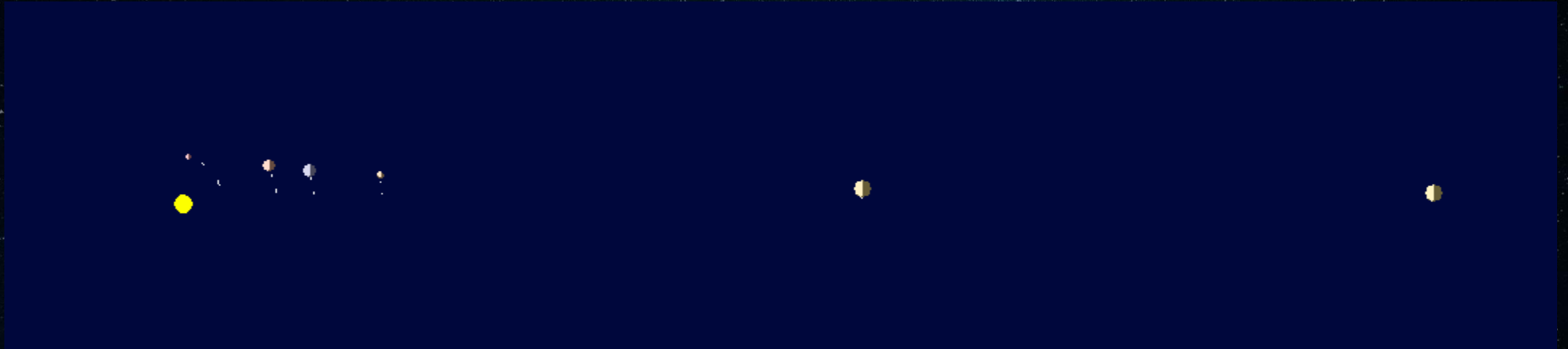
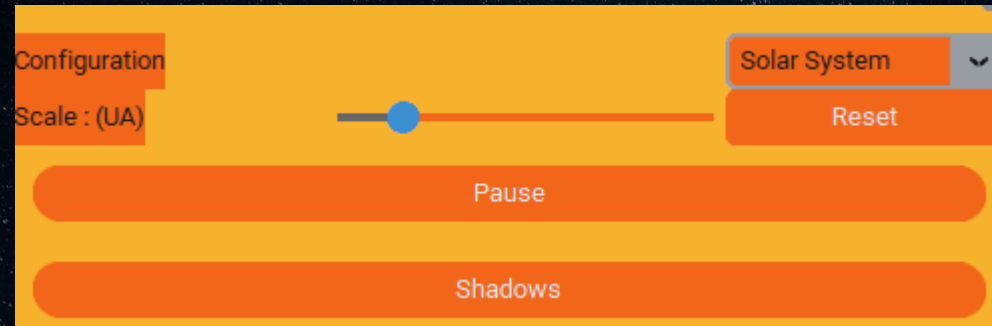
Speed (vx, vy) (m/s):

Default

Add

Cancel

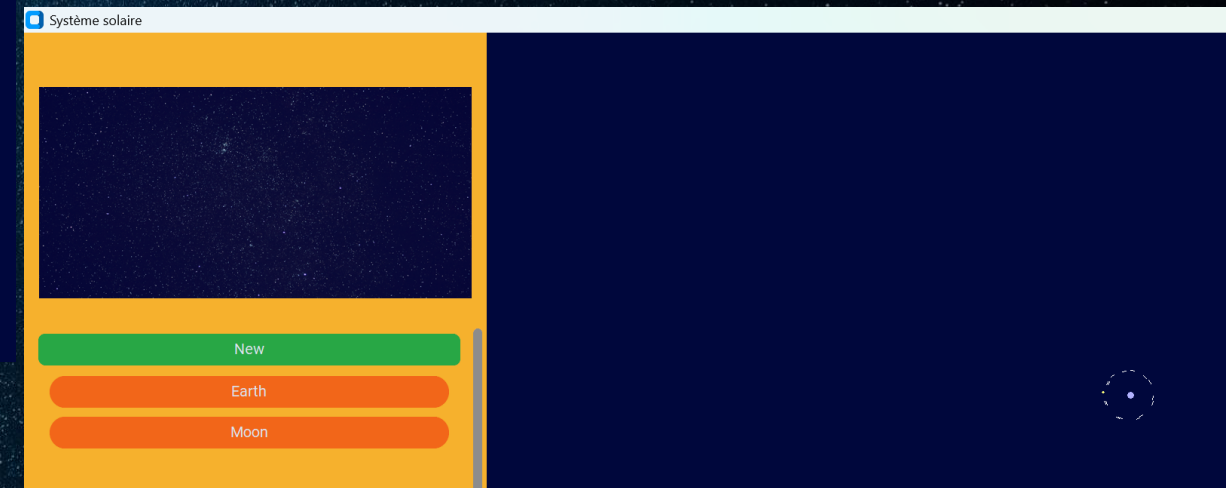
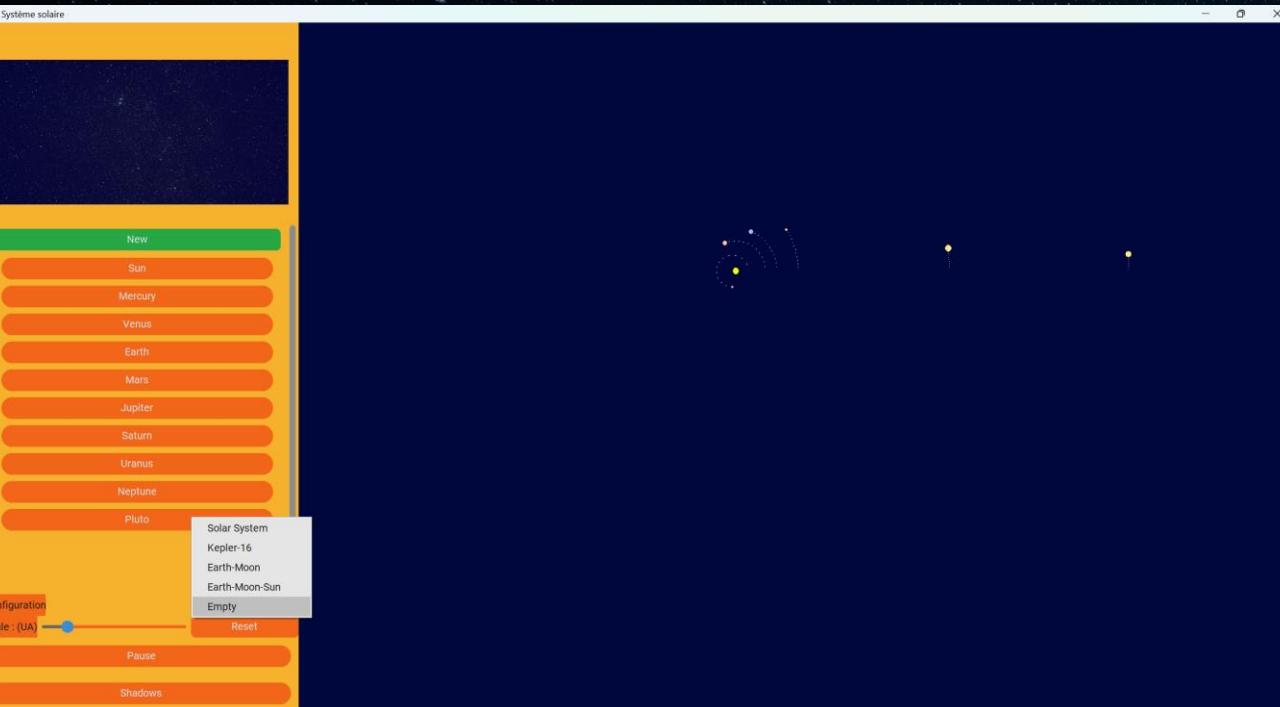
Windows



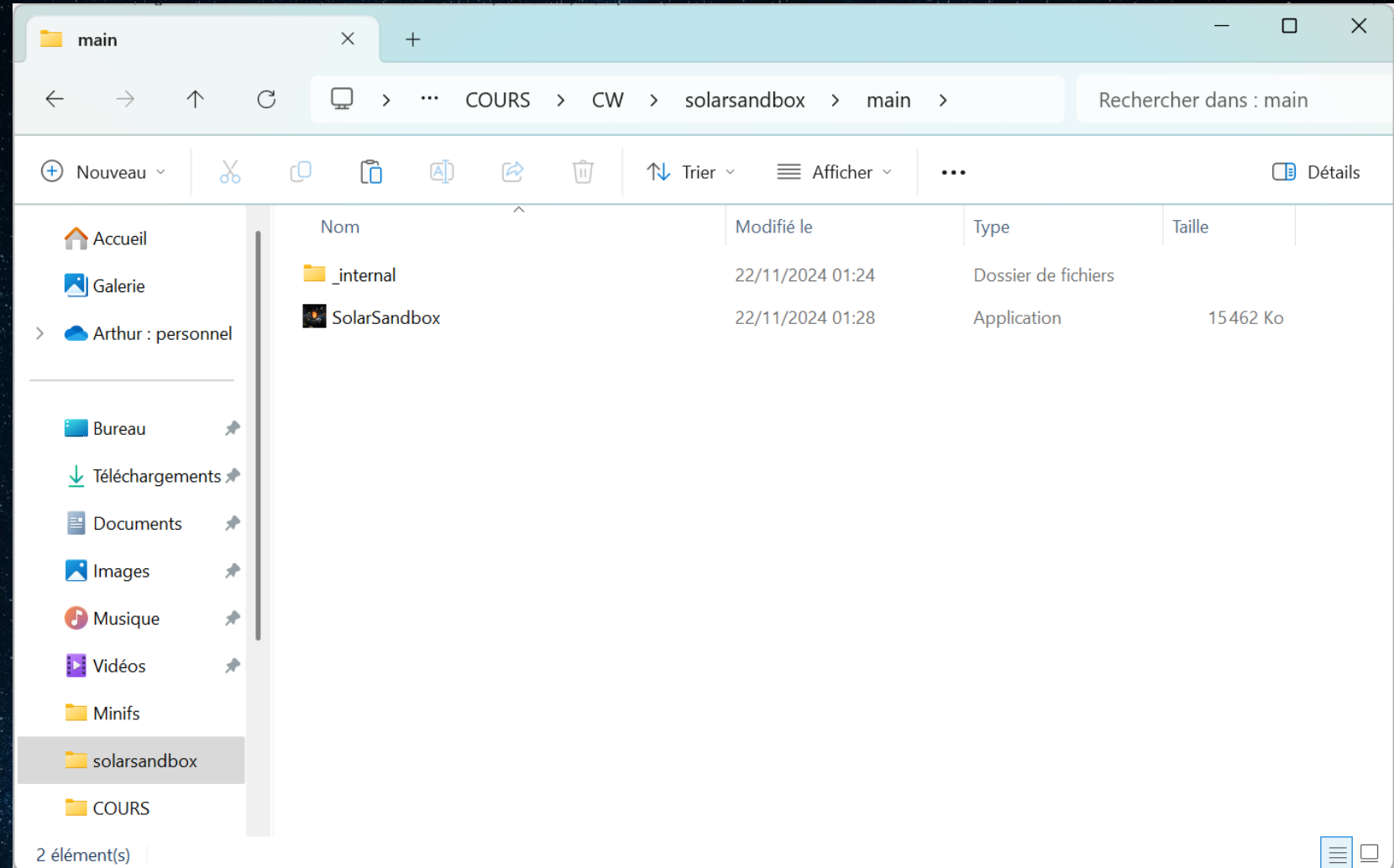
It is also possible :

- to remove celestial bodies with "Suppr"
- to move the background with the mouse
- to select a star by clicking on it in the simulation

Preset configurations



Making a .exe with Pyinstaller



Easier using auto-py-to-exe

It's Demo time !